

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

November 2024

Database development with PL/SQL

Reference reading



1. [Improve your PL/SQL skills with quizzes, workouts, and classes](#)
2. [PL SQL Exercises](#)

Lecture 08 – Exercises Authid

Scenario: Employee Data Access and Role-Based Permissions in an Employee Management System

In a PL/SQL-based Employee Management System, you can implement a scenario using `authid` and `current_user` to define and manage employee roles, permissions, and interactions securely within the database. Here's an example scenario:

1. System Overview

The Employee Management System stores sensitive information about employees, including personal data, salary, and performance. The system is accessible to different users with distinct roles such as `HR_Manager`, `Employee`, and `Admin`. Each role has specific permissions for data access and operations.

2. Requirement

Only users with `HR_Manager` and `Admin` roles should have access to sensitive employee details, such as salary and performance metrics. Regular employees can only view their own information.

3. Implementation Using `AUTHID` and `CURRENT_USER`

PL/SQL procedures and functions can use `AUTHID` and `CURRENT_USER` to enforce role-based access control. The `AUTHID` clause allows a procedure to run either with the permissions of the definer (the user who created the procedure) or the invoker (the user who executes it).

4. Code Setup

- **Define Roles:** Set up roles for `HR_Manager`, `Employee`, and `Admin`.
- **Create Procedures and Functions:** Create PL/SQL procedures to manage and fetch employee data securely.
- **`AUTHID` and `CURRENT_USER`:** Use `AUTHID CURRENT_USER` to make sure the procedure checks the permissions of the user executing it.

Scenario: Solution with AUTHID CURRENT_USER

```
-- Define roles
CREATE ROLE HR_Manager;
CREATE ROLE Employee;
CREATE ROLE Admin;

-- Grant permissions to roles
GRANT SELECT, UPDATE ON Employees TO HR_Manager;
GRANT SELECT ON Employees TO Employee;
GRANT ALL ON Employees TO Admin;

-- Create an Employees table for the demonstration
CREATE TABLE Employees (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100),
    emp_role VARCHAR2(50),
    emp_salary NUMBER,
    emp_performance VARCHAR2(50)
);

-- Insert sample data
INSERT INTO Employees VALUES (1, 'Alice', 'HR_Manager', 60000, 'Excellent');
INSERT INTO Employees VALUES (2, 'Bob', 'Employee', 40000, 'Good');
INSERT INTO Employees VALUES (3, 'Carol', 'Admin', 80000, 'Outstanding');

-- Procedure to fetch employee data, using AUTHID CURRENT_USER for role-based access
CREATE OR REPLACE PROCEDURE Get_Employee_Data (emp_id_in NUMBER)
AUTHID CURRENT_USER
```

```
AS
    emp_name VARCHAR2(100);
    emp_salary NUMBER;
    emp_performance VARCHAR2(50);
BEGIN
    -- Check if CURRENT_USER has the HR_Manager or Admin role
    IF USER IN ('HR_Manager', 'Admin') THEN
        -- If user has HR_Manager or Admin role, allow access to all data
        SELECT emp_name, emp_salary, emp_performance INTO emp_name, emp_salary, emp_performance
        FROM Employees
        WHERE emp_id = emp_id_in;

        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_name);
        DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp_salary);
        DBMS_OUTPUT.PUT_LINE('Employee Performance: ' || emp_performance);

    ELSIF USER = 'Employee' THEN
        -- If user is an Employee, allow access only to their own data
        IF emp_id_in = Get_Current_Employee_ID() THEN
            SELECT emp_name INTO emp_name FROM Employees WHERE emp_id = emp_id_in;
            DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_name);
        ELSE
            DBMS_OUTPUT.PUT_LINE('Access Denied: You can only view your own data.');
```



SQL*P

Scenario – Explanation

Explanation

1. **AUTHID CURRENT_USER:** This procedure is defined with AUTHID CURRENT_USER, meaning it will execute with the permissions of the user who calls it, not the user who defined it. This allows the procedure to enforce role-based restrictions based on the caller's role.
2. **Role-based Access Control:**
 - If CURRENT_USER has the HR_Manager or Admin role, they can access all details of any employee.
 - If CURRENT_USER is an Employee, they can only view their own data. An attempt to access other employees' data results in an "Access Denied" message.
3. **CURRENT_USER Variable:** This setup allows the procedure to dynamically check who is executing it and provide or deny access accordingly.

Expected Outputs

- **Admin or HR_Manager Calls** Get_Employee_Data(1): Full data about Alice is displayed.
- **Employee Bob Calls** Get_Employee_Data(2): Bob can view his own data.
- **Employee Bob Calls** Get_Employee_Data(1): Bob is denied access since he is trying to access another employee's data.

Scenario: Employee Data Access with AUTHID DEFINER

In a PL/SQL Employee Management System, using **AUTHID DEFINER** instead of AUTHID CURRENT_USER makes the procedure run with the privileges of the procedure's *definer* (the user who created the procedure) rather than the caller (or current user). This approach allows a centralized control over who can execute the procedure but can impact role-based access management.

Scenario with AUTHID DEFINER

In this scenario, we'll show how **AUTHID DEFINER** can be used to manage access when a single user (like an Admin) defines procedures that other users (like employees) can call but with specific access restrictions hardcoded within the procedure.

1. System Overview

The Employee Management System handles employee data, which is accessible to various roles. Only HR_Manager and Admin users can access sensitive details, but employees can view their basic information.

2. Requirement

The procedure Get_Employee_Data should allow only employees with specific permissions to access or modify certain data fields. Using AUTHID DEFINER ensures that any user executing the procedure does so under the privileges of the definer, who controls all permissions.

3. Implementation Using AUTHID DEFINER

- Define a procedure as AUTHID DEFINER, which means it will execute with the definer's permissions.
- Use conditions in the procedure to manage role-specific access to sensitive data.

Solution - AUTHID DEFINER

```
-- Define the procedure with AUTHID DEFINER
CREATE OR REPLACE PROCEDURE Get_Employee_Data (emp_id_in NUMBER)
AUTHID DEFINER
AS
    emp_name VARCHAR2(100);
    emp_salary NUMBER;
    emp_performance VARCHAR2(50);
    emp_role VARCHAR2(50);
BEGIN
    -- Retrieve the role of the employee based on emp_id_in
    SELECT emp_role INTO emp_role FROM Employees WHERE emp_id = emp_id_in;

    IF emp_role = 'HR_Manager' OR emp_role = 'Admin' THEN
        -- Allow full access for HR_Manager and Admin
        SELECT emp_name, emp_salary, emp_performance INTO emp_name, emp_salary,
emp_performance
        FROM Employees
        WHERE emp_id = emp_id_in;

        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_name);
        DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp_salary);
        DBMS_OUTPUT.PUT_LINE('Employee Performance: ' || emp_performance);

    ELSIF emp_role = 'Employee' THEN
        -- Allow access to limited information for regular employees
        SELECT emp_name INTO emp_name
        FROM Employees
        WHERE emp_id = emp_id_in;

        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_name);
        DBMS_OUTPUT.PUT_LINE('Restricted Access: Salary and Performance data is not accessible.');
```

```
ELSE
    DBMS_OUTPUT.PUT_LINE('Access Denied: Unauthorized role.');
```

```
END IF;
END Get_Employee_Data;
```

Scenario – Explanation

Explanation

1. **AUTHID DEFINER:** This procedure will always run with the privileges of the definer, such as an Admin user. The users who call this procedure, regardless of their individual privileges, will run it as if they were the definer, enabling centralized permission control.
2. **Role Check Logic:**
 - Inside the procedure, a check is made based on the employee's role (emp_role).
 - If the emp_role is HR_Manager or Admin, full data access (including salary and performance) is provided.
 - If the emp_role is Employee, only the employee's name is shown, with salary and performance data access restricted.
 - If the user has an unauthorized role, an "Access Denied" message is returned.

Benefits of AUTHID DEFINER

- **Centralized Access Control:** The procedure's access control is based on the definer's permissions. This is useful in systems where a single user or a small set of users control which procedures can be executed by others.
- **Simpler Permission Management:** Only the definer needs to have permissions on underlying tables, and they can define exactly what each role can see or do.
- **Minimized User Privilege Requirements:** Users do not need specific table privileges; they just need EXECUTE privileges on the procedure.

Expected Outputs

- **Admin or HR_Manager Calls** Get_Employee_Data(1): Full employee data, including salary and performance, is displayed.
- **Employee Calls** Get_Employee_Data(2): Only the employee's name is displayed, and salary/performance data is restricted.
- **Unauthorized Role:** Displays "Access Denied: Unauthorized role."

In summary, using AUTHID DEFINER centralizes the access control around the definer's permissions, which allows for more rigid and simplified access management.

Keep Learning!